EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Mathematics and Computer Science
Software Engineering and Technology Research Group

Configurable Monitoring Infrastructure for ROS 2 Applications

Master's Thesis

Shikai Jin
Student ID: 2086360

Supervisors:
Dr. Ivan Kurtev

Draft version

Eindhoven, June 2026

Abstract

Preface

Contents

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Context and Motivation

Robotic systems increasingly combine sensing, planning, control, communication, and interaction with a physical environment. Their behaviour is therefore not determined by software alone: it also depends on hardware, other systems, and conditions that may only become known during operation. This makes exhaustive design-time verification difficult and creates a continuing need to observe deployed robots. Runtime monitoring addresses this need by collecting information from an executing system and checking relevant functional or non-functional properties [18, 10].

The Robot Operating System 2 (ROS 2) is a widely used software platform for building such systems. It encourages applications to be decomposed into nodes that communicate through typed interfaces. Topics carry continuous data streams, services implement request-response interactions, and actions model long-running tasks with feedback and results [11, 15]. This component-based style helps developers reuse and replace functionality, but it also spreads a robot's behaviour across many independently executing nodes. Relevant evidence may be distributed over several interfaces and may only be meaningful when related over time.

ROS 2 offers useful observation mechanisms, including normal subscriptions, graph discovery, service introspection, hidden action interfaces, topic statistics, and lower-level tracing. However, turning those mechanisms into a project-specific runtime monitor still requires substantial engineering. A developer must decide what to observe, reduce high-volume messages to relevant fields, preserve source and timing context, deliver the observations to a checking engine, and publish or store the resulting verdicts. These decisions also vary between use cases. A monitor used during local development may run entirely on the robot, while an operational deployment may keep data collection near the robot and execute more expensive checking on a separate machine.

The preparation phase of this project approached these choices as variability in the domain of ROS 2 monitoring. Its initial feature model distinguished, among others, observable elements, centralized and distributed architectures, active and passive modes, single- and multi-robot systems, and different implementation paradigms. The current prototype develops a focused subset of that broad domain: passive observation of topics, service events, and action feedback or status; configurable processing of captured data; pluggable property-specific checking; and both integrated and split deployments.

1.2 Problem Statement

Existing runtime-monitoring approaches for robotics demonstrate that ROS data can be intercepted, translated into formal traces, checked against properties, or visualized. They also reveal several recurring tensions.

First, the observable surface of ROS 2 is heterogeneous. A topic can be observed by adding a subscriber, but service communication is opaque unless introspection is enabled, and an action is implemented using a combination of hidden topics and services. A monitoring infrastructure that only handles ordinary topics misses interactions that can be central to robot behaviour.

Second, useful observation is application dependent. Recording every field of every message can consume unnecessary storage and bandwidth. Conversely, discarding too much data prevents later analysis. The required balance changes per source and per property: a pose stream may need rate limiting, a plan may only be relevant when it changes, and a navigation action may require selected progress fields.

Third, observation and property evaluation have different concerns. Data collection is tied to ROS 2 interface semantics, while property evaluation may use a threshold rule, a state machine, Linear Temporal Logic (LTL), Signal Temporal Logic (STL), or a project-specific domain-specific language (DSL). Binding the infrastructure to one formalism limits reuse; embedding all property details in the collector makes changes risky and difficult to test.

Fourth, the location of monitoring work is a deployment decision. Running all stages on the robot is simple and avoids a network dependency, but consumes robot-side resources. Moving evaluation away from the robot reduces that load and separates concerns, but introduces transport, failure, ordering, and correlation issues. A useful infrastructure should support these deployment choices without requiring a different monitoring model for each one.

The problem addressed by this thesis is therefore:

> How can a configurable and extensible monitoring infrastructure observe heterogeneous ROS 2 interactions, prepare those observations for property-specific evaluation, and support multiple deployment topologies without coupling the infrastructure to one verification formalism?

1.3 Research Objective and Questions

The objective of this thesis is to design, implement, and evaluate a configuration-driven monitoring infrastructure for ROS 2 applications. The infrastructure should offer reusable mechanisms for collecting and routing runtime data, while allowing use-case-specific monitoring semantics to be added as plugins. It should also make the boundary between robot-side observation and verifier-side evaluation explicit, so that an equivalent monitoring pipeline can be deployed locally or split across machines.

This objective is refined into four research questions:

RQ1 Which common and variable capabilities are relevant when configuring runtime monitoring for ROS 2 applications?

RQ2 How can topics, service events, and action progress be represented and processed through a common observation pipeline while retaining sufficient source, session, and timing context?

RQ3 How can the infrastructure decouple data collection, transport, property-specific conversion, verdict evaluation, and verdict delivery so that these concerns remain extensible and deployable in different topologies?

RQ4 To what extent does the resulting prototype provide the intended functionality in representative integrated and split deployments, and what performance and operational trade-offs does it introduce?

RQ1 connects the project to the domain-analysis perspective established during the preparation phase. RQ2 and RQ3 concern the architecture and its implementation. RQ4 guides the evaluation using controlled examples and a ROS 2 Navigation (Nav2) case study.

1.4 Approach

The work follows an iterative design-and-build approach. Literature on software monitoring and ROS-based runtime verification is used to identify recurring responsibilities and variation points. These are compared with ROS 2's communication mechanisms and with practical monitoring scenarios. The resulting architecture is then instantiated as a Python prototype and refined through unit tests, containerized deployment examples, and experiments with Nav2.

The prototype organizes monitoring as a sequence of explicit stages:

1. A collector observes a configured ROS 2 topic, service event stream, or action feedback/status stream.

2. The observation is wrapped in a uniform data record containing the payload and contextual information such as source type, source name, message type, timestamps, session identifier, and record identifier.

3. A per-source transformer pipeline selects fields or suppresses unneeded records, for example by throttling a high-frequency stream or emitting only when selected values change.

4. Dispatchers route records to one or more exporters and independently to configured conversion-and-verdict chains.

5. A data converter translates infrastructure-level records into the representation expected by a property-specific verdict service.

6. The verdict service evaluates the converted record and emits a structured verdict when appropriate. Verdict exporters then deliver the result to files, standard output, MQTT, or user-defined destinations.

The same conceptual pipeline supports multiple deployments. In an integrated deployment, all stages execute in one ROS 2-facing process. In a split deployment, a monitor process collects and transforms records on the robot side, an MQTT broker transports them, and a verifier-side runner performs conversion and verdict evaluation without requiring ROS 2. The transport boundary is expressed through generic *Source* and *Exporter* abstractions rather than being embedded in the checking logic.

1.5 Scope

The broad monitoring domain described during the preparation phase contains more capabilities than can be addressed in one thesis. The implemented and evaluated scope is deliberately constrained.

1.5.1 In Scope

- Passive observation of ROS 2 topics, service request/response events, and action feedback/status streams.
- YAML-based configuration of monitored sources, transformations, output transports, converter/verdict chains, and split deployment inputs.
- A common record representation for heterogeneous ROS 2 observations, including identifiers that support correlation between input records and emitted verdicts.
- Per-source preprocessing through field extraction, rate throttling, and change filtering.
- Pluggable converters, verdict services, inbound sources, and outbound exporters selected through importable class names.
- File and MQTT delivery of monitoring records, and file, standard output, and MQTT delivery of verdicts.
- Integrated robot-side and split robot/verifier deployment modes, demonstrated with Docker Compose examples.
- Functional validation through unit tests and representative ROS 2 and Nav2 scenarios.

1.5.2 Out of Scope

- A generic parser or execution engine for a particular temporal logic or DSL. Such semantics belong to pluggable verdict services.
- Active enforcement, automatic recovery, or feedback into the robot. The prototype observes and reports but does not change application behaviour.
- Complete observation of all action phases. Feedback and status are supported directly; goal and result interactions depend on service introspection and are not completed as a unified action trace.
- Monitoring parameters, node lifecycle state, DDS-level QoS events, or network packets.
- Guarantees of global message ordering, hard real-time execution, Byzantine fault tolerance, or exactly-once delivery across distributed deployments.
- A graphical configuration interface, dashboard, persistent query service, or automatic monitor-code generation.

These boundaries are important when comparing the work with related systems. For example, ROS-FM targets efficient network-level monitoring and security policy enforcement [19], while `ros2_tracing` targets low-overhead tracing of ROS 2 internals [2]. The prototype in this thesis instead focuses on application-level observations and the configurable path from those observations to extensible verdict services.

1.6 Contributions

The expected contributions of this thesis are:

1. A focused characterization of common and variable capabilities for a ROS 2 runtime-monitoring infrastructure, informed by a feature-oriented domain view and existing monitoring architectures.
2. An architecture that separates ROS 2 observation, record transformation, routing, property-specific conversion, verdict evaluation, and result delivery.
3. A configuration model and prototype that provide a common observation path for topics, service events, and action feedback/status.
4. A plugin model that keeps property semantics and transport choices outside the monitoring core.
5. Deployment support for both an all-in-one ROS 2 monitor and a split robot/verifier arrangement connected through a pluggable transport.
6. An evaluation of the prototype through automated tests, deployment demonstrations, and a Nav2 monitoring case study.

The main contribution is not a new temporal logic or verdict algorithm. It is the infrastructure that allows different property evaluators and transports to be connected to ROS 2 observations with limited framework-specific code.

1.7 Thesis Outline

Chapter 2 introduces ROS 2 communication and observability, runtime monitoring and verification concepts, domain variability, and deployment concerns. Chapter 3 compares existing ROS monitoring, verification, tracing, and deployment approaches and positions this thesis among them. Chapter 4 will present the architecture and design decisions. Chapter 5 will describe the prototype. Chapter 6 will evaluate the implementation, and Chapter 7 will discuss its limitations and implications. Chapter 8 will conclude the thesis and identify future work.

Chapter 2

Background

This chapter introduces the concepts needed to understand the monitoring infrastructure developed in this thesis. It first describes the ROS 2 execution model and the communication mechanisms that form the observable surface of an application. It then discusses runtime monitoring and runtime verification, generic monitoring responsibilities, variability in monitoring systems, and the deployment concerns that arise when observation and checking are separated.

2.1 ROS 2 Applications

2.1.1 Nodes and the ROS Graph

ROS 2 is a middleware and software ecosystem for developing robotic applications. Its architecture supports modular systems built from reusable components and provides communication, discovery, configuration, build, and development tooling [11]. Although its name contains "operating system," ROS 2 does not replace a conventional operating system. Instead, it provides abstractions and tools above the operating system and communication middleware.

An executing ROS 2 application is represented by the *ROS graph*. The graph contains nodes and the communication relationships between them. A node is a process-level or component-level participant that performs a focused responsibility, such as publishing sensor observations, estimating pose, planning a route, or controlling motion. Nodes may execute on the same machine or on different machines and may dynamically join or leave the graph.

ROS 2 communication is strongly typed. An interface definition specifies the fields exchanged through a topic, service, or action. Client libraries such as `rclcpp` and `rclpy` generate language-specific types and APIs from these definitions. The type information is important for monitoring: captured bytes are only useful at the application level when they can be associated with and deserialized according to the correct interface type.

The distributed graph creates both an opportunity and a challenge for monitoring. Adding an observer can often be done without modifying the application node, but no single application callback necessarily exposes the complete behaviour of the robot. The monitor must select relevant graph interfaces and preserve enough context to relate observations to their source.

2.1.2 Topics

Topics implement anonymous publish-subscribe communication. Publishers send typed messages under a topic name, and any number of subscribers may receive them. Publishers and subscribers communicate without directly naming each other. Topics are intended for continuous data streams such as sensor readings, estimated state, actuator commands, or diagnostic information [15, 14].

Topics are naturally observable because a monitoring node can create an additional subscription. The observer does not need to be inserted between the original publisher and subscriber, and the existing data path can continue unchanged. Tools such as `ros2 topic echo` and `ros2 bag record` use this principle.

This convenience does not remove all monitoring decisions. A topic can publish large messages at a high rate, and several publishers may share one topic name. The monitor must choose an appropriate Quality of Service profile, determine which fields matter, decide whether every message must be retained, and record which source nodes were visible. Furthermore, subscribing observes messages as received by the monitor; it does not by itself reconstruct a globally ordered history across several distributed publishers.

2.1.3 Services

Services implement typed request-response communication. A client sends a request to a named service and receives a response from its server. Services are intended for short operations that produce an immediate result, such as a query or a bounded calculation [15].

Unlike topics, service interactions are not observable by simply adding another ordinary client. By default, an unrelated node cannot see when a service was called or inspect its request and response. ROS 2 therefore provides *service introspection*. When a client or server enables introspection, it publishes event messages containing metadata and, depending on the configured mode, the request or response contents. The introspection states range from disabled, through metadata-only, to full contents [12].

A monitor can subscribe to the service event topic and derive a record whose phase is either request or response. This offers a non-invasive observation mechanism, but it depends on the application enabling introspection. Monitoring infrastructure must therefore distinguish between the service's logical name and its hidden event topic, and it must retain event metadata needed to correlate requests and responses.

2.1.4 Actions

Actions represent long-running, goal-oriented behaviour. An action client sends a goal to an action server, receives progress feedback while the goal is executing, may request cancellation, and eventually receives a result. Actions are suitable for operations such as navigation or manipulation that take time and may need to be preempted [15, 3].

An action is not one primitive communication channel. ROS 2 implements actions using hidden services and topics. Goal, result, and cancellation interactions use services, while feedback and status use topics. Hidden names allow ROS 2 tools to recognize these interfaces as the implementation of one logical action [3].

For monitoring, this means that action observation is inherently multi-phase. A subscriber can directly observe feedback and status streams. Observing goals, results, and cancellations requires the corresponding service interactions to be introspectable. A monitor must also report the logical action name rather than exposing only its hidden implementation topics, otherwise downstream analysis becomes unnecessarily coupled to ROS 2 internals.

2.1.5 Discovery, Middleware, and Quality of Service

ROS 2 uses a middleware abstraction layer so that different implementations can provide the underlying communication. Common implementations are based on the Data Distribution Service (DDS) standard. DDS supplies decentralized discovery and data-centric publish-subscribe communication, avoiding the single ROS master used by ROS 1 [11].

Discovery allows a node to inspect currently visible topic and service names and their types. A monitor can use this information to reduce configuration effort, for example by discovering a type when only a source name is provided. Discovery is asynchronous, however. A node or type may not yet be visible when the monitor starts, and graph membership can change later. Consequently, startup-time discovery is useful but not equivalent to a complete dynamic model of the system.

ROS 2 also exposes Quality of Service (QoS) policies. Policies control behaviour such as reliability, durability, queue history, and deadlines. Compatible profiles allow communication to occur under different assumptions, ranging from best-effort streaming to reliable delivery [13]. A monitoring subscription therefore participates in the communication semantics; an incompatible profile may prevent observations, while a queue that is too small may lose data under load. QoS event monitoring is a valuable but separate capability from application-message monitoring.

2.2 Runtime Monitoring and Runtime Verification

2.2.1 Terminology

Software monitoring is the observation and checking of properties or quality attributes of an executing system [18]. The term covers a broad range of activities, including resource monitoring, performance monitoring, requirements monitoring, tracing, intrusion detection, and runtime verification. These activities share infrastructure concerns but differ in the information they collect and the questions they answer.

Runtime verification (RV) is a lightweight verification technique that analyzes an observed execution against a specification [10]. In contrast to exhaustive design-time techniques such as model checking, RV reasons about the execution trace that actually occurred. It can detect that an observed behaviour satisfies or violates a property, but it cannot generally prove correctness for all possible future executions.

This thesis uses the following terms:

System under monitoring The ROS 2 application whose behaviour is observed.

Observation or event A captured runtime occurrence, such as a topic message, service request, service response, or action feedback message.

Trace An ordered or partially ordered sequence of observations used for analysis.

Property An expected condition over one observation or a trace.

Monitor A component or collection of components that observes the system and evaluates or supports evaluation of properties.

Verdict A structured result produced by property evaluation, for example that a property holds or that a violation occurred.

2.2.2 Online and Offline Monitoring

Online monitoring processes observations while the system is executing. It can provide timely notifications and may support recovery or adaptation. Its cost is also paid during operation, so latency, resource usage, and isolation from the system under monitoring matter.

Offline monitoring evaluates recorded traces after execution. It cannot react immediately, but it can use more expensive analysis, replay the same trace against several properties, and avoid adding the full checking cost to the running robot. A monitoring infrastructure may support both modes by using a stable record format and separating collection from evaluation.

The current project primarily targets online data flow, but file-based JSON Lines output creates a boundary that can later support offline replay. This illustrates why persistence and transport should be separated from the semantics of a particular property.

2.2.3 Passive and Active Monitoring

A passive monitor observes and reports without modifying application behaviour. An active monitor can intervene, for example by blocking a message, triggering a recovery action, changing a configuration, or switching to a safe controller. Active monitoring can provide runtime assurance, but it also becomes part of the control path and therefore demands stronger guarantees.

The prototype developed in this thesis is passive. It may run alongside the application or on a separate verifier host, and it emits verdicts for other components or people to consume. Active response is treated as future work.

2.2.4 Centralized, Decentralized, and Distributed Monitoring

A centralized monitoring arrangement sends observations to one checking location. It simplifies property evaluation because relevant state can be maintained in one place, but it may create a bottleneck or single point of failure. A decentralized monitor decomposes checking among several local monitors, while a distributed system may have both the monitored processes and the monitoring computation spread across locations.

Distributed runtime verification introduces questions that do not arise in a single process: clocks may differ, messages may be delayed or reordered, communication may fail, and observation can perturb the system. Classification dimensions include whether one global monitor exists, whether a shared clock is assumed, whether monitors tolerate failures, and whether they use a model of the system's distribution [7].

The split deployment in this thesis is distributed in an engineering sense: collection and checking may run on separate hosts connected by MQTT. It does not claim a distributed-verification algorithm, global event ordering, or failure-aware verdict semantics. The verifier evaluates the record order delivered by its configured source.

2.3 Responsibilities of a Monitoring Infrastructure

The monitoring literature uses different terms for similar responsibilities. Instrumentation, interception, probing, and data extraction may all describe how information is acquired. Domain analysis helps reveal these commonalities and separates them from choices that vary between monitoring solutions [18].

2.3.1 Information Collection

Information collection connects to the system under monitoring and creates observations. In ROS 2 this can involve subscriptions, service introspection, hidden action topics, graph queries, tracing hooks, or network capture. The collection mechanism determines which events are visible and how much it perturbs the application.

Filtering may already occur during collection. For example, a monitor may sample a topic at a maximum rate or retain only selected fields. Early reduction saves bandwidth and

storage but permanently removes data. It should therefore be configurable per source and kept distinct from property evaluation.

2.3.2 Transformation and Enrichment

Raw observations often require transformation before analysis. A nested ROS 2 message can be projected to selected fields, normalized, aggregated, or translated into events understood by a DSL engine. Context may also be added, such as timestamps, source identifiers, sequence numbers, or session identifiers.

The distinction between infrastructure-level and property-level transformation is useful. Infrastructure-level transformation reduces or normalizes observations without knowing the full property semantics. Property-level conversion creates the exact representation expected by a verdict engine. For example, a general field extractor can retain `twist.linear.x`, while a property-specific converter can rename it to speed and attach a property identifier.

2.3.3 Routing, Distribution, and Persistence

After collection, observations must reach their consumers. They may be written to a local file, published over a message broker, sent to a database, or fanned out to several destinations. Routing also determines where analysis occurs. Rabiser et al. identify routing, monitoring-information transformation, and checking as central responsibilities of a monitoring processing layer [18].

Persistence supports debugging, audit, replay, and later analysis. JSON Lines is a practical format for append-only records because each line is an independent JSON value. A record should contain a discriminator and stable identifiers so that consumers can distinguish data events from session bookends and link verdicts back to their evidence.

2.3.4 Checking and Verdict Production

Checking determines whether observed data satisfies a property. A checker may perform a simple comparison, maintain a finite-state machine, evaluate an LTL or STL formula, detect a statistical anomaly, or invoke an external oracle. The required input representation and state model depend on the selected formalism.

Verdict semantics also vary. A checker may emit a Boolean result for every input, report only violations, report transitions into and out of a violation, or produce a quantitative robustness value. RTAMT, for example, supports quantitative robustness monitoring for STL, where the magnitude and sign of a value express how strongly a signal satisfies or violates a property [22]. The prototype's example threshold services use edge-triggered Boolean verdicts: one verdict is emitted when a threshold breach begins and another when it clears.

2.3.5 Application and Reaction

The result of checking may be visualized, stored, sent to an operator, consumed by a test harness, or used for active adaptation. Treating verdict delivery as another routing concern allows a property service to remain independent of the final reaction. This is especially useful when the same verdict must be written to a file and streamed to a dashboard or external controller.

2.4 Variability and Configuration

2.4.1 Feature-Oriented Domain Analysis

Feature-Oriented Domain Analysis (FODA) is a method for discovering and representing commonalities and differences among related systems. It models user-visible characteristics as mandatory, optional, or alternative features and uses them to describe a family of possible products [9].

A feature model is useful for monitoring because there is no single ideal monitoring product. One deployment may observe only topics and write local files; another may observe service interactions, reduce data on the robot, evaluate properties remotely, and publish verdicts to several consumers. Representing these choices explicitly helps distinguish a stable architecture from configurable variation points.

The preparation-phase feature model for this thesis identified broad variation in observable elements, monitoring architecture, monitoring mode, system configuration, and implementation paradigm. The implemented prototype selects a practical subset of those features and expresses many of them through YAML.

2.4.2 Declarative Configuration and Plugins

Declarative configuration describes *what* components and settings should be used without hard-coding the complete wiring procedure in application code. In the prototype, a YAML document declares monitored sources, transformer chains, exporters, converter/verdict chains, and verifier-side inputs.

Configuration alone cannot anticipate every future DSL or transport. A plugin mechanism complements it by allowing a configured type to refer to an importable class. This creates two levels of extensibility:

- Built-in short names cover common infrastructure components, such as a file exporter, MQTT exporter, field extractor, or rate throttler.
- Fully qualified class names select use-case-specific converters, verdict services, sources, or exporters without modifying the framework registry.

This design keeps the core focused on lifecycle, data flow, configuration, and error isolation. Property authors retain control over their input schema and evaluation semantics.

2.5 Deployment and Transport

2.5.1 Integrated Monitoring

In an integrated deployment, collection, transformation, conversion, checking, and verdict delivery execute in one process or on one host. This topology has few moving parts and avoids an external transport between observation and checking. It is appropriate for development, small applications, or properties that are inexpensive to evaluate.

Its main drawback is resource coupling. Collection and checking consume CPU, memory, storage, and network capacity on the same platform as the robot application. A failed or expensive plugin may also affect the monitoring process close to the robot.

2.5.2 Split Monitoring

In a split deployment, a robot-side process observes and reduces ROS 2 data, then exports records to a verifier-side process. The verifier can run without ROS 2 if it consumes the

stable record representation rather than ROS message objects. This supports independent deployment and can move expensive checking away from constrained robot hardware.

The split introduces a transport boundary. Records may be delayed, duplicated, lost, or delivered after a disconnection. The monitor and verifier also need separate session identifiers, while verdicts should preserve the identity of the originating monitor session and input records. These concerns motivate explicit record and verdict schemas.

2.5.3 MQTT as a Transport

MQTT is a lightweight publish-subscribe messaging protocol commonly used for machine-to-machine and Internet of Things communication. A broker routes messages published under hierarchical topic names to interested subscribers. In the prototype, MQTT is one pluggable transport rather than a fixed part of the monitoring semantics.

Robot-side data-record exporters publish observations to topic names derived from their source type and source name. A verifier-side MQTT source subscribes to a topic filter and reconstructs data records before forwarding them into the same conversion-and-verdict pipeline used by the integrated deployment. Verdicts can independently be published to another MQTT topic. This symmetry illustrates the value of treating outbound destinations as exporters and inbound transports as sources.

2.6 Design Implications

The background concepts lead to several implications for a configurable ROS 2 monitoring infrastructure:

1. The infrastructure needs source-specific collectors because topics, services, and actions expose different observation mechanisms.
2. Heterogeneous observations should enter a common record model so that downstream transport and routing do not depend on ROS message classes.
3. Early transformation should be configurable per source to control data volume, while property-specific conversion should remain separate.
4. Checking must be formalism agnostic at the infrastructure level.
5. Routing and fan-out should isolate failures so that one output does not prevent other outputs or property chains from receiving records.
6. Integrated and split deployments should reuse the same converter and verdict wiring.
7. Distributed deployment should make its limitations explicit, especially regarding ordering, delivery guarantees, and clocks.

These implications provide the conceptual basis for the architecture developed in the later design chapter. Before presenting that architecture, Chapter 3 examines how existing approaches address different parts of the monitoring problem.

Chapter 3

Related Work

Runtime monitoring for robotic systems is studied from several perspectives. Some approaches focus on intercepting ROS communication, some generate monitors from formal requirements, some provide a configurable data-collection platform, and others trace low-level execution or network behaviour. This chapter reviews the approaches most relevant to the thesis and positions the project according to its primary goal: configurable infrastructure from ROS 2 observation to extensible verdict delivery.

3.1 Comparison Dimensions

Direct comparison is difficult because "monitoring" refers to different activities across the literature. A tracing framework that explains callback latency and a runtime-verification framework that detects a temporal-property violation both observe execution, but they answer different questions. The following dimensions are used to structure the comparison.

Observation level Whether the approach observes application messages, generated events, network packets, middleware events, or operating-system execution.

ROS interface coverage Whether topics, services, and actions are supported, and whether support is native to ROS 2.

Property semantics Whether checking is tied to one formalism, delegates to an external oracle, or is deliberately left pluggable.

Configuration and extensibility Whether users can select sources, fields, rates, transports, checking engines, and outputs without changing the framework core.

Deployment Whether observation and checking can be placed independently, and whether the work specifically addresses distributed monitoring concerns.

Primary objective Whether the approach prioritizes formal assurance, flexible data collection, performance diagnosis, security, visualization, or verification deployment.

These dimensions reflect the broader monitoring reference architecture described by Rabiser et al., in which information collection, processing, routing, checking, persistence, and final application are related but separable responsibilities [18].

3.2 ROSMonitoring

Ferrando et al. introduced ROSMonitoring as a runtime-verification framework for ROS applications [6]. Its central idea is to insert generated monitor nodes into selected topic communication paths. A YAML configuration describes which topics to intercept and how monitor nodes should behave. The monitor translates intercepted messages into events and sends them to an *oracle*, which evaluates a formal specification. This separation makes ROSMonitoring formalism agnostic: the interception framework is not limited to one property language.

ROSMonitoring is closely related to this thesis in three ways. First, both approaches treat configuration as a way to reduce manual integration work. Second, both separate ROS-facing observation from a property evaluator. Third, both aim to avoid coupling the monitoring infrastructure to one formalism.

The approaches differ in how they observe and route data. ROSMonitoring instruments an application by remapping communication so that monitor nodes are placed between publishers and subscribers. This enables online interception and can control whether messages are republished. The prototype in this thesis uses passive subscriptions and introspection streams, leaving the original communication path unchanged. It also makes transformation, record export, verdict export, and the boundary between integrated and split deployment explicit.

The original ROSMonitoring work focuses on topic communication in ROS 1. ROSMonitoring 2.0 extends the framework with service monitoring and with mechanisms that account for the order in which messages are published and received [8]. Some service-monitoring functionality is ported to ROS 2, while full ROS 2 support and action monitoring are described as future work. Ordering is a particularly important contribution because a monitor receiving several topics cannot assume that reception order equals publication order.

This thesis does not solve the ordering problem addressed by ROSMonitoring 2.0. It instead supports ROS 2 topics, introspected service events, and action feedback/status through a uniform record pipeline. The resulting systems are therefore complementary: ROSMonitoring provides stronger interception and ordering mechanisms for runtime verification, while this work emphasizes a configurable ROS 2 data path and pluggable deployment/-transport boundaries.

3.3 ROMoSu

Stadler, Vierhauser, and Cleland-Huang identify recurring challenges in establishing runtime monitoring for ROS-based applications: understanding the system structure, selecting data, collecting at appropriate frequencies, performing diverse constraint checks, and providing useful insights [21]. Their preliminary architecture aims to reduce setup and maintenance effort through a configurable framework.

ROMoSu develops this direction into a flexible runtime-monitoring framework [20]. It distinguishes a monitoring-configuration phase from a runtime data-collection phase. Users create reusable monitoring configurations that select topics and subtopics, choose publication frequencies, and associate configurations with types of systems under monitoring. ROMoSu includes a framework core, an administration user interface, a monitoring dashboard, configuration persistence, and extension points for external services. Its evaluation covers simulated and physical ROS-based systems.

ROMoSu is the closest related approach with respect to configuration and variation. Both projects recognize that monitoring needs change between applications and scenarios, and both provide source selection, data reduction, and external checking or service integration. ROMoSu offers a broader user-facing monitoring platform, including configuration management and visualization. The current thesis prototype is smaller in product scope but

more explicit about the internal data-flow abstractions used to compose collectors, transformers, converters, verdict services, sources, dispatchers, and exporters.

Another difference is the emphasis on ROS 2 interface coverage and deployment symmetry. The prototype treats topics, service events, and action feedback/status as first-class source categories. It also uses the same converter/verdict chain in an integrated ROS 2 process or in a verifier-side runner that has no ROS 2 dependency. ROMoSu motivates flexible monitoring configurations, while this thesis investigates a compact plugin-oriented infrastructure for composing and relocating monitoring stages.

3.4 Generated Runtime Monitors for ROS 2

Perez et al. present an end-to-end workflow for generating ROS 2 runtime monitors from structured natural-language requirements [16]. Requirements are specified in NASA's Formal Requirement Elicitation Tool (FRET), translated into temporal-logic formulae, converted by Ogma into Copilot monitor specifications, and compiled into hard-real-time C99. An Ogma backend generates self-contained ROS 2 packages that receive application data and publish violations.

This approach addresses an important source of monitoring errors: manually implementing a monitor from an informal requirement. It provides a traceable path from requirement elicitation to generated executable checking code and prioritizes predictable monitor execution. In comparison, the infrastructure in this thesis does not generate property monitors and cannot claim that a plugin faithfully implements an original requirement. Instead, it provides the observation, transformation, routing, and deployment substrate into which a generated or manually written verdict service could be integrated.

The two approaches therefore operate at different layers. FRET–Ogma–Copilot strengthens the path from requirements to correct monitor code. This thesis focuses on making runtime observations and transport choices available to different checkers. A future integration could wrap generated monitors behind the verdict-service interface, combining stronger requirement traceability with the configurable data pipeline.

3.5 RTAMT and Temporal-Logic Checking

RTAMT is a runtime-monitoring library for Signal Temporal Logic (STL) specifications [22]. It supports online and offline monitoring, discrete- and dense-time interpretations, and quantitative robustness semantics. It also provides integration examples for ROS 2 and MATLAB/Simulink. Rather than returning only a Boolean answer, robustness values express the degree to which an observed signal satisfies or violates a property.

RTAMT illustrates why the infrastructure should not prescribe one DSL-record shape. An STL engine needs timestamped signals and maintains temporal state, while a threshold checker may need only a single named value. The *DataConverter* and *VerdictService* abstractions in this project are intended to accommodate such differences. An RTAMT adapter could convert selected data records into timestamped signal updates and expose robustness results as structured verdicts.

Compared with this thesis, RTAMT provides substantially richer formal checking semantics but does not aim to be a general ROS 2 observation and transport framework. The present prototype includes only simple example verdict services; its contribution is the configurable path into and out of a checker such as RTAMT.

3.6 Tracing and Performance Monitoring

`ros2_tracing` provides multipurpose, low-overhead tracing of ROS 2 internals [2]. It instruments the ROS 2 core and integrates with Linux Trace Toolkit: next generation (LTTng), allowing developers to analyze callback execution, message flow, latency, and operating-system events. The work is designed for real-time systems and evaluates the overhead of its instrumentation.

This level of observation is valuable when high-level messages do not explain why deadlines are missed or where performance bottlenecks occur. It is also different from the application-level property monitoring targeted by this thesis. The current prototype observes deserialized interface data and prepares it for functional or use-case-specific checks. It does not trace executor internals, callback durations, memory allocation, or middleware handling.

The approaches can be combined. A functional verdict may indicate that a robot violated a behavioural property, while a synchronized low-level trace may help diagnose the cause. The data-record model and verdict correlation identifiers in this thesis provide a starting point for relating such evidence, but direct trace integration remains future work.

3.7 Network-Level Monitoring and Security

ROS-FM uses extended Berkeley Packet Filter (eBPF) and eXpress Data Path (XDP) technology to monitor ROS network traffic efficiently [19]. It adds modules for security-policy enforcement and distributed visualization and reports lower overhead than generic tools in larger ROS systems. Its observation point is below ROS application interfaces, which allows it to address performance and security questions across ROS 1 and ROS 2 network traffic.

The network-level approach avoids creating an ordinary ROS subscriber for each observed topic and can detect or enforce communication policies efficiently. It also requires protocol-aware processing to recover application meaning and is less directly aligned with service introspection or property-specific message conversion. The prototype in this thesis accepts the cost and semantics of application-level ROS 2 observation in exchange for typed, deserialized data and straightforward source configuration.

ROS intrusion-detection datasets such as ROSIDS23 and ROSPaCe further show the importance of observations across multiple architectural layers [5, 17]. ROSPaCe, for example, combines operating-system, network, and ROS 2 service features. Such systems target security detection rather than general property-checking infrastructure, but they reinforce the value of pluggable transports and analysis components.

3.8 Containerized and Distributed Verification

Aldegheri et al. propose a containerized ROS-compliant verification environment in which monitors synthesized from LTL assertions are encapsulated as plug-and-play ROS nodes [1]. The work addresses where monitors should execute across edge-to-cloud layers so that verification accuracy and real-time constraints can be balanced. Docker provides portability and supports runtime management of verification resources.

This work is related to the split deployment in the present prototype. Both recognize that monitor placement is a first-class decision and that checking may need to move away from resource-constrained robot hardware. The containerized verification environment focuses on orchestration and placement of generated LTL monitors under resource and latency constraints. This thesis provides a simpler engineering mechanism: robot-side records are exported over a configured transport and consumed by a ROS-independent verifier runner. It does not optimize placement or provide real-time guarantees.

More generally, distributed runtime verification research studies monitor decomposition, clocks, failure models, and the relationship between the distribution of the system and the monitor [7]. The prototype should not be confused with these algorithms. Its split mode supports distributed deployment, but checking remains logically centralized at each verdict runner and follows delivered record order.

3.9 Runtime Verification and Field-Based Testing Guidance

Caldas et al. provide guidelines for architects, developers, and quality assurance teams applying runtime verification and field-based testing to ROS-based systems [4]. Their work emphasizes that real-world environments cannot be fully reproduced in simulation and that robotic software should be designed to facilitate telemetry, isolation, monitoring reactions, and field evidence.

This thesis contributes an implementation-oriented building block aligned with that guidance. The record schema provides telemetry and correlation; exporters support local or remote evidence collection; plugin boundaries isolate property logic from collection; and split deployment can separate the ROS-facing monitor from checking. The prototype does not cover the complete field-testing lifecycle, test generation, recovery strategies, or assurance argument.

3.10 Summary Comparison

Table 3.1 summarizes the primary relationship between selected approaches and this thesis. A "Partial" entry means that the capability is limited to a subset, prototype port, or indirect integration.

Table 3.1: Comparison of selected related work.

Approach	Primary focus and observation	Checking and deployment	Relationship to this thesis
ROSMonitoring / 2.0	Runtime verification over intercepted ROS messages and service interactions; partial ROS 2 support	External, formalism-agnostic oracle; YAML instrumentation; online/offline; ordering support in 2.0	Closest in formalism-agnostic interception; this thesis uses passive ROS 2 collection and explicit transport/verdict plugins.
ROMoSu	Flexible collection of selected ROS topics and subtopics	External constraint services; strong configuration, reusable scenarios, UI, and dashboard	Closest in flexible configuration; this thesis emphasizes compact composable pipeline abstractions and split verifier deployment.
FRET–Ogma–Copilot	Generation of ROS 2 monitor packages from structured requirements	Generated hard-real-time monitors in self-contained packages	Stronger requirement-to-monitor path; could be integrated as a verdict plugin.

Approach	Primary focus and observation	Checking and deployment	Relationship to this thesis
RTAMT	Runtime verification of timestamped numeric signals	Online/offline STL with quantitative robustness; ROS 2 integration example	Provides rich checker semantics that the thesis intentionally leaves pluggable.
`ros2_tracing`	Low-overhead ROS 2 core and operating-system tracing	Trace analysis and orchestration integration rather than property verdicts	Complementary for performance diagnosis below application-level records.
ROS-FM	Efficient ROS 1/ROS 2 network observation through eBPF/XDP	Metrics, security-policy enforcement, and visualization	Complementary network/security layer; this thesis uses typed application data.
Containerized verification environment	ROS topic observations consumed by generated monitors	Generated LTL assertion monitors with edge-to-cloud placement and Docker orchestration	Related split deployment goal; this thesis offers pluggable source/exporter boundaries without placement optimization.
This thesis	Typed ROS 2 topics, introspected service events, and action feedback/status	Pluggable converter/verdict services; YAML pipeline; integrated and split deployment	Focuses on reusable infrastructure from observation through verdict delivery.

3.11 Research Gap

The reviewed work provides strong solutions for individual parts of the monitoring problem. ROSMonitoring offers formalism-agnostic interception and advances service and ordering support. ROMoSu provides reusable monitoring configurations and user-facing management. Generated-monitor approaches create a traceable path from requirements to executable verification. RTAMT provides rich temporal checking, while `ros2_tracing` and ROS-FM provide low-level performance and network visibility. Containerized verification work addresses monitor placement across computational layers.

The gap addressed by this thesis is a compact, ROS 2-native infrastructure that connects heterogeneous application-level observations to extensible checking and output components while preserving the same conceptual pipeline across integrated and split deployments. Specifically, the intended combination is:

- passive collection from topics, service introspection events, and action progress interfaces;
- a uniform record with source, timing, session, and evidence identity;
- configurable per-source reduction before storage or transport;
- independent routing of records to outputs and property chains;

- property-specific conversion and verdict services loaded as plugins;
- pluggable inbound and outbound transports around a ROS-independent verifier runner; and
- structured verdict fan-out and correlation to input records.

This gap is intentionally practical rather than absolute: several related systems could be extended to provide overlapping capabilities. The thesis investigates whether these responsibilities can be represented through a small set of composable abstractions and configured without making ROS 2 collection, transport, and property semantics depend on each other.

Chapter 4

Design

Chapter 5

Implementation

Chapter 6

Evaluation

Chapter 7

Discussion

Chapter 8

Conclusions

Bibliography

[1] Stefano Aldegheri, Nicola Bombieri, Samuele Germiniani, Federico Moschin, and Graziano Pravadelli. A containerized ROS-compliant verification environment for robotic systems. In *2021 Design, Automation and Test in Europe Conference and Exhibition*, pages 222–225, 2021. 18

[2] Christophe Bédard, Ingo Lütkebohle, and Michel R. Dagenais. `ros2_tracing`: Multipurpose low-overhead framework for real-time tracing of ROS 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022. 4, 18

[3] Geoffrey Biggs, Jacob Perron, and Shane Loretz. Actions. ROS 2 Design, 2019. Accessed 2026-06-06. 8

[4] Ricardo Caldas, Juan Antonio Piñera García, Matei Schiopu, Patrizio Pelliccione, Genaína Rodrigues, and Thorsten Berger. Runtime verification and field-based testing for ROS-based robotic systems. *IEEE Transactions on Software Engineering*, 50(10):2544–2567, 2024. 19

[5] Elif Değirmenci, Yunus Sabri Kırca, İlker Özçelik, and Ahmet Yazıcı. ROSIDS23: Network intrusion detection dataset for robot operating system. *Data in Brief*, 51:109739, 2023. 18

[6] Angelo Ferrando, Rafael C. Cardoso, Michael Fisher, Davide Ancona, Luca Franceschini, and Viviana Mascardi. ROSMonitoring: A runtime verification framework for ROS. In *Towards Autonomous Robotic Systems*, volume 12228 of *Lecture Notes in Computer Science*, pages 387–399. Springer, 2020. 16

[7] Adrian Francalanza, Jorge A. Pérez, and César Sánchez. Runtime verification for decentralised and distributed systems. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 176–210. Springer, 2018. 10, 19

[8] Maryam Ghaffari Saadat, Angelo Ferrando, Louise A. Dennis, and Michael Fisher. ROSMonitoring 2.0: Extending ROS runtime verification to services and ordered topics. In *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems*, volume 411 of *Electronic Proceedings in Theoretical Computer Science*, pages 38–55, 2024. 16

[9] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, 1990. 12

[10] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. 1, 9

[11] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. 1, 7, 9

[12] Open Robotics. Configure service introspection. ROS 2 Documentation. Accessed 2026-06-06. 8

[13] Open Robotics. Quality of service settings. ROS 2 Documentation. Accessed 2026-06-06. 9

[14] Open Robotics. Ros 2 topics. ROS 2 Documentation. Accessed 2026-06-06. 8

[15] Open Robotics. Topics vs services vs actions. ROS 2 Documentation. Accessed 2026-06-06. 1, 8

[16] Ivan Perez, Anastasia Mavridou, Tom Pressburger, Alexander Will, and Patrick J. Martin. Monitoring ROS2: From requirements to autonomous robots. In *Proceedings of the Fourth Workshop on Formal Methods for Autonomous Systems*, volume 371 of *Electronic Proceedings in Theoretical Computer Science*, pages 208–216, 2022. 17

[17] Tommaso Puccetti, Simone Nardi, Cosimo Cinquilli, Tommaso Zoppi, and Andrea Ceccarelli. ROSPaCe: Intrusion detection dataset for a ROS2-based cyber-physical system and IoT networks. *Scientific Data*, 11(1):481, 2024. 18

[18] Rick Rabiser, Klaus Schmid, Holger Eichelberger, Michael Vierhauser, Sam Guinea, and Paul Grünbacher. A domain analysis of resource and requirements monitoring: Towards a comprehensive model of the software monitoring domain. *Information and Software Technology*, 111:86–109, 2019. 1, 9, 10, 11, 15

[19] Sean Rivera, Antonio Ken Iannillo, Sofiane Lagraa, Clément Joly, and Radu State. ROSFM: Fast monitoring for the robotic operating system (ROS). In *2020 25th International Conference on Engineering of Complex Computer Systems*, pages 15–24, 2020. 4, 18

[20] Marco Stadler and Michael Vierhauser. ROMoSu: Flexible runtime monitoring support for ROS-based applications. In *2023 IEEE/ACM 5th International Workshop on Robotics Software Engineering*, pages 53–60, 2023. 16

[21] Marco Stadler, Michael Vierhauser, and Jane Cleland-Huang. Towards flexible runtime monitoring support for ROS-based applications. In *Proceedings of the 4th International Workshop on Robotics Software Engineering*, pages 12–15. ACM, 2022. 16

[22] Tomoya Yamaguchi, Bardh Hoxha, and Dejan Ničković. RTAMT: Runtime robustness monitors with application to CPS and robotics. *International Journal on Software Tools for Technology Transfer*, 26(1):79–99, 2024. 11, 17

Appendix A

Appendix